

Event Sourcing Interview Quick Notes

30 backend/system-design concepts in short, revision-friendly language

Remember the core flow

Command -> Aggregate -> Event Store -> Event -> Broker -> Consumer -> Projection

Event	Something happened.
Command	Please do something.
Event Sourcing	History is the source of truth.
CQRS	Write model and read model are separate.
Idempotency	Duplicate message, same final result.
Outbox	DB write + event record committed together.
Saga	Distributed transaction as multiple coordinated steps.

Interview rule: Do not say Event Sourcing = Kafka. Event Sourcing is a persistence pattern; Kafka is a messaging/streaming platform.

Best one-line answer: Event Sourcing stores immutable domain events and rebuilds current state by replaying them.

Concepts 1-5

1. Event

A fact that already happened in the system. Usually named in past tense.

Example: *OrderCreated, PaymentCompleted, UserRegistered.*

2. Command

A request to make something happen. It can succeed or fail.

Example: *CreateOrder -> if valid -> OrderCreated.*

3. Event-Driven Architecture

Services communicate by publishing and reacting to events instead of calling every service directly.

Example: *OrderCreated -> Payment, Inventory, Email consumers react.*

4. Producer

A service that creates and publishes an event.

Example: *Order Service publishes OrderCreated.*

5. Consumer

A service that listens for an event and performs work.

Example: *Email Service consumes OrderCreated and sends confirmation.*

Concepts 6-10

6. Message Broker

The middle system that carries events from producers to consumers.

Example: Kafka, RabbitMQ, AWS SNS/SQS, Google Pub/Sub.

7. Event Sourcing

Store the sequence of events as the source of truth, not only the latest state.

Example: Deposit +1000, Withdraw -200 -> current balance = 800.

8. Event Store

A database/log that permanently stores an aggregate's events in order.

Example: Account-101: Opened -> Deposited -> Withdrawn.

9. Aggregate

A business object that protects rules and decides whether a command can produce an event.

Example: BankAccount rejects WithdrawMoney(500) if balance is only 100.

10. Event Replay

Re-apply old events to rebuild state or rebuild a projection.

Example: Replay all Order events to recreate an analytics database.

Concepts 11-15

11. Snapshot

Saved aggregate state at a certain version so you do not replay every old event.

Example: Snapshot at event 10,000 + replay only events 10,001 onward.

12. Projection

A read-friendly view built from events.

Example: OrderCreated + OrderPaid -> read table shows status = PAID.

13. CQRS

Separate writes (commands) from reads (queries), often with different models/databases.

Example: Write to Event Store; read order status from PostgreSQL/Redis.

14. Eventual Consistency

Different parts of the system may be briefly out of sync, but become consistent later.

Example: Order is written now; read model updates 50 ms later.

15. Idempotency

Processing the same event twice should have the same final effect as processing it once.

Example: If eventId already processed, ignore the duplicate.

Concepts 16-20

16. Event Ordering

Events for the same entity must be processed in the correct sequence when order matters.

Example: OrderCreated must happen before OrderShipped.

17. At-Least-Once Delivery

The broker tries not to lose messages, but a message may be delivered more than once.

Example: Design consumers to handle duplicates safely.

18. Dead-Letter Queue (DLQ)

A place for messages that still fail after retries.

Example: Bad Payment event fails 5 times -> move to DLQ for investigation.

19. Retry / Backoff

Retry temporary failures, waiting longer between attempts to avoid hammering a failing service.

Example: Retry after 1s, then 2s, then 4s.

20. Schema Evolution

Change event structure without breaking old consumers or historical events.

Example: Add a new optional field instead of suddenly removing an old one.

Concepts 21-25

21. Event Versioning

Give events versions so consumers know which shape/meaning they are reading.

Example: UserRegistered v1 -> later UserRegistered v2.

22. Optimistic Concurrency

Write only if the aggregate version is still what you originally read.

Example: Expected version 5, actual version 6 -> reject/retry.

23. Transactional Outbox

Write business data and an outbox event in one DB transaction, then publish the outbox later.

Example: INSERT order + INSERT outbox in one COMMIT; worker sends to Kafka.

24. Dual-Write Problem

Updating the database and publishing an event separately can leave them inconsistent if one succeeds and one fails.

Example: DB saved order, Kafka publish failed -> other services never hear about it.

25. Consumer Lag

How far a consumer is behind the newest available event.

Example: Producer writes 10k events/sec; consumer handles 8k/sec -> lag grows.

Concepts 26-30

26. Saga

Coordinate a multi-service business transaction using steps and events instead of one big DB transaction.

Example: Order -> Payment -> Inventory -> Shipping.

27. Compensating Transaction

An action that logically undoes an earlier successful step when a later step fails.

Example: Inventory failed after payment -> issue RefundPayment.

28. Correlation ID

One shared ID used to trace all messages belonging to the same request/workflow.

Example: req-123 appears in OrderCreated, PaymentCompleted, InventoryReserved.

29. Observability

Use logs, metrics and traces to understand what the distributed system is doing.

Example: Track consumer lag, retry count, DLQ size, latency and errors.

30. When NOT to Use Event Sourcing

Avoid it when simple CRUD is enough and audit/replay/history do not justify the extra complexity.

Example: Simple todo/admin app usually needs normal CRUD, not Event Sourcing.

Last-minute revision: Know the definition, one example, one benefit, and one trade-off. That is enough to answer most first-round interview questions clearly.